A*
Entegrasyonu

Ceren Adıyaman
16/12/2025

Table of Contents

1. Amaç ve Kapsamı 2
2.1 Mimari 2
2.2 Yol Ağı (Graph) Yönetimi 3
2.3 Edge-Based Routing Yaklaşımı 4
2.4 Gerçek Konum ve Virtual Node Mantığı 4
2.5 Routing 4
2.6 Rota Hesaplama Akışı 4
3. A* Rota Entegrasyon Katmanı 5
3.1 Hedef Tipine Göre Ayrışan Davranış 5
4. SUIT Frontend 6
4.1 RouteOptimizationMap – Rota Verisinin Görselleştirilmesi 6
4.2 RouteVehicleSelector – Araç Seçimi ve A* Tetikleme Mantığı 6
5. OPEVA Frontend 8
5.1 Araç Seçimi ve Hedef Tercihi Mantığı 9
5.2 Araç Konumuna Bağlı A* Tetikleme Kararı 10
5.3 A* İsteğinin Oluşturulması ve Gönderilmesi 10
5.4 Route4Vehicle.json Güncellemesi ve Senkronizasyon 10

1. Amaç ve Kapsamı

Bu çalışma, mevcut rota optimizasyon mimarisine, araçların gerçek zamanlı konumlarından başlayarak rota oluşturabilmesini sağlayan A* tabanlı bir hesaplama yapısının entegre edilmesini amaçlamaktadır. Sistemin önceki halinde tüm rotalar yalnızca depo başlangıçlı olarak üretilmekteydi. Ancak gerçek saha senaryolarında araçlar her zaman depoda bulunmayabilir. Bu nedenle rota motorunun, FIWARE üzerinden alınan son araç konumlarını dikkate alacak şekilde güncellenmesi gerekliliği ortaya çıkmıştır.

2.Route Service (Routing Microservice)

Route Service, araçların gerçek zamanlı konumlarından hedef noktaya (depo veya müşteri) giden en uygun rotayı hesaplayan bağımsız bir mikroservistir.

Bu servis:

- SUMO simülasyonuna bağımlı olmadan çalışır
- OSMnx + NetworkX tabanlı yol grafiğini kullanır
- Gerçek araç yönü (heading) ve konumu dikkate alır

2.1 Mimari

app.py: Flask tabanlı Route Service'i başlatır ve API endpoint'lerini ayağa kaldırır.

config.py: Servis genelinde kullanılan yapılandırma ve dosya yolu ayarlarını içerir.

database.py: FIWARE veritabanından araçların güncel ve geçmiş konum bilgilerini alır.

graph_manager.py: Yol ağını GraphML cache, SUMO net dosyası veya OSMnx üzerinden oluşturur ve yönetir.

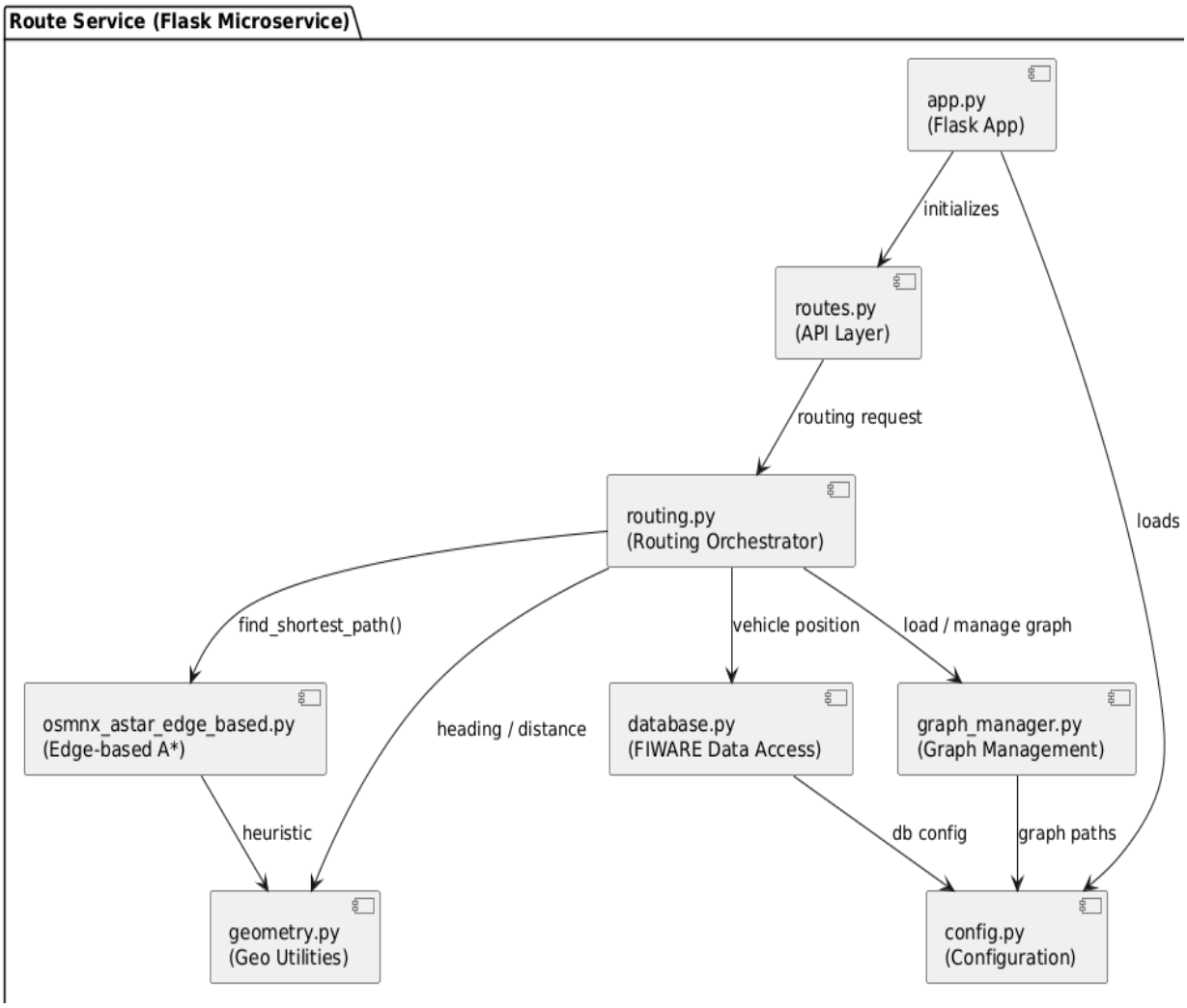
osmnx_astar_edge_based.py: Edge-based A* algoritmasını ve heuristic hesaplamalarını içerir.

geometry.py: Haversine mesafesi, heading ve temel coğrafi hesaplamaları gerçekleştirir.

routing.py: Routing sürecini uçtan uca koordine eder; virtual node ve A* çağrılarını yönetir.

routes.py: API endpoint'lerini tanımlar ve routing isteklerini servis içine yönlendirir.

Şekil 1 Component Diagram



2.2 Yol Ağı (Graph) Yönetimi

Routing işlemlerinin temelini yol ağı (graph) oluşturmaktadır. Bu graph, öncelikli olarak daha önce oluşturulmuş bir GraphML cache dosyasından yüklenir. Cache bulunmaması durumunda, eğer sistemde SUMO net dosyası mevcutsa, bu dosya NetworkX MultiDiGraph formatına dönüştürülerek kullanılır. Her iki seçenek de yoksa, OSMnx kullanılarak OpenStreetMap üzerinden yol ağı indirilir ve daha sonra GraphML olarak kaydedilir. Bu yaklaşım, servisin hem offline hem online senaryolarda çalışabilmesini sağlar.

Graph yapısı özellikle MultiDiGraph olarak korunur. Bunun nedeni, aynı iki node arasında birden fazla edge bulunabilmesi ve SUMO tarafındaki lane ve edge bilgilerinin kaybolmamasıdır. Edge'ler üzerinde hız, uzunluk, seyahat süresi, edge_id ve lane_id gibi bilgiler saklanır.

2.3 Edge-Based Routing Yaklaşımı

Bu projede klasik node-based routing yaklaşımı yerine edge-based A* algoritması tercih edilmiştir. Node-based routing, özellikle kavşaklarda aracın geliş yönünü dikkate almadığı için gerçekçi olmayan rota seçimlerine yol açabilmektedir. Edge-based yaklaşımda ise her bir edge (u, v, key) ayrı bir durum (state) olarak ele alınır ve rota, edge'ler üzerinden hesaplanır. Bu sayede aracın hangi yönden geldiği ve hangi yöne devam ettiği açıkça kontrol edilebilir.

A* algoritmasında kullanılan heuristic fonksiyon, edge'lerin geometrik merkezleri arasındaki Haversine mesafesine dayanmaktadır (lat lon üzerinden hesaplamada daha iyi olduğu için). Bu mesafe, graph üzerindeki maksimum izin verilen hız değeri ile bölünerek tahmini seyahat süresine dönüştürülür. Böylece heuristic hem admissible kalır hem de zaman ağırlıklı routing için uygun bir alt sınır sağlar.

2.4 Gerçek Konum ve Virtual Node Mantığı

Gerçek araçlar genellikle bir yolun tam ortasında veya kavşak node'larının üzerinde bulunmaz. Bu nedenle routing işlemi doğrudan mevcut node'lardan başlatılamaz. Bu problemi çözmek için araç ve hedef konumlar için graph üzerine geçici (virtual) node'lar eklenir. Bu virtual node'lar, konuma en yakın edge'lere bağlanarak graph yapısına dahil edilir.

Eğer araç için bir heading bilgisi mevcutsa, bu bilgi başlangıç edge'inin seçilmesinde kullanılır. Böylece aracın ters yöne dönmesi veya fiziksel olarak mümkün olmayan ilk hamleler yapması engellenir. Heading bilgisi yoksa veya güvenilir değilse, sistem otomatik olarak daha genel bir başlangıç edge'i seçerek fallback davranışı sergiler.

2.5 Routing

Routing işlemi sırasında servis öncelikle araç konumunu ve gerekiyorsa geçmiş konum bilgilerini veritabanından alır. Araç hareket ediyorsa, son iki farklı konumdan heading hesaplanır. Daha sonra başlangıç ve hedef için virtual node'lar eklenir ve edge-based A* algoritması çalıştırılır. Başarılı bir rota bulunması durumunda, edge dizisi node dizisine dönüştürülür ve rota üzerindeki tüm noktalar coğrafi koordinatlara çevrilir.

2.6 Rota Hesaplama Akışı

1.Araç konum bilgisi alınır.
Gelen istekte vehicle_id, latitude ve longitude bilgileri değerlendirilir.

- GET isteklerinde araçların en güncel konumu MySQL veritabanındaki vehicle_tracking_fiware tablosundan alınır.
- POST isteklerinde konum bilgisi doğrudan request body üzerinden okunur.
- Araç hareket hâlindeyse, önceki konum verileri kullanılarak heading (yön) bilgisi hesaplanır. **Routing yaklaşımı belirlenir.**
  use_edge_based parametresi true ise (varsayılan), heading bilgisini destekleyen **edge-based A*** algoritması kullanılır. Bu değer false olduğunda sistem node-based routing yaklaşımına geçer.

2. Virtual node'lar oluşturulur.
Araç ve hedef (depo veya müşteri) konumları için graph üzerine geçici (virtual) node'lar eklenir. Bu node'lar, gerçek konumlara en yakın edge'lere bağlanır. Heading bilgisi mevcutsa, başlangıç

için yönle uyumlu edge choose_edge_by_bearing() fonksiyonu ile seçilir ve 180° ters yönlü başlangıçlar engellenir.

3. Başlangıç ve hedef edge'ler belirlenir.
Başlangıç edge'i, araç virtual node'undan çıkan ve heading ile uyumlu edge olarak tanımlanır. Hedef edge ise hedef virtual node'una bağlanan edge olarak belirlenir.

4.*Edge-based A algoritması çalıştırılır*.
Cache'lenmiş OSMnxEdgeAStar instance'ı kullanılarak find_shortest_path() fonksiyonu çağrılır.

- **Ağırlık (cost):** Seyahat süresi (travel_time, saniye)
- **Heuristic:** Haversine mesafesi / 50 km/s
  Bulunan edge dizisi, node dizisine dönüştürülerek rota elde edilir.

5. Fallback mekanizması uygulanır.
Edge-based routing başarısız olursa, sistem fallback olarak node-based routing'e geçer. Bu durumda en yakın node'lar belirlenir ve Haversine mesafesine dayalı node-based A* algoritması çalıştırılarak tahmini bir rota üretilir.

6. Rota çıktısı oluşturulur.
Elde edilen node veya edge path, route_to_latlon() fonksiyonu ile coğrafi koordinatlara (lat/lon) dönüştürülür. Rota için toplam mesafe, tahmini süre ve edge sayısı hesaplanarak sonuç döndürülür.

3. A* Rota Entegrasyon Katmanı

backend/routes/astar.js dosyası, Python tabanlı Route Service tarafından hesaplanan A* rotalarının, sistemin kullandığı Route4Vehicle.json yapısına entegre edilmesinden sorumludur. Bu katman, routing algoritmasının kendisini çalıştırmaz; bunun yerine, hesaplanmış rotayı alır, bağlamına göre yorumlar ve simülasyon ile operasyonel sistemlerin kullanabileceği hale getirir. Böylece routing hesaplaması ile rota verisinin senaryo dosyalarına işlenmesi birbirinden ayrılmış olur.

3.1 Hedef Tipine Göre Ayrışan Davranış

Hedef tipi first_customer olarak belirlendiğinde, sistem Route4Vehicle.json dosyasında ilgili rotanın start_point yapısını tamamen yeniden oluşturur. Bu durumda aracın gerçek zamanlı konumu start_point.location alanı olarak kaydedilir ve A* algoritmasıyla hesaplanan rota start_point.waypoints listesi altında saklanır. Ayrıca node_detail alanı altında bir sonraki hedef müşteri bilgisi eklenerek, aracın artık depo yerine operasyonel olarak ilk müşteriye doğru hareket ettiği açıkça ifade edilir.

Hedef tipi depot olduğunda ise daha korumalı bir güncelleme yaklaşımı izlenir. A* ile hesaplanan rota, mevcut start_point.waypoints listesine eklenir veya mevcut yapı ile birleştirilir. Bu sayede Route4Vehicle.json dosyasındaki depo merkezli senaryo akışı korunur ve yalnızca aracın mevcut konumundan depoya olan başlangıç segmenti güncellenmiş olur. Bu yaklaşım özellikle SUIT senaryosunda tercih edilir; çünkü tüm rotalar kavramsal olarak depo başlangıçlı ve depo hedefli olacak şekilde tanımlanmıştır.

4. SUIT Frontend

4.1 RouteOptimizationMap – Rota Verisinin Görselleştirilmesi

Harita bileşeni, Route4Vehicle.json verisini yükledikten sonra, rota üzerindeki tüm koordinatları toplayarak harita sınırlarını otomatik olarak ayarlar. Böylece kullanıcı, manuel yakınlaştırma yapmadan tüm rotaları tek ekranda görebilir. Eğer bir rotanın start_point alanında astar_applied bilgisi bulunuyorsa, bu rota A* algoritmasıyla güncellenmiş bir başlangıç noktası olarak işaretlenir ve başlangıç ikonu buna göre değiştirilir. Bu durum, kullanıcının hangi rotaların dinamik olarak hesaplandığını ayırt edebilmesini sağlar.

SUIT tarafında rota sadece depoya doğru tanımlı olduğu için, başlangıç ve bitiş noktaları konsept olarak depo merkezlidir. Ancak araç depoda değilse, start_point konumu araçtan depoya giden A* rotasıyla güncellenmiş şekilde gösterilir. Bu yaklaşım, SUIT senaryosunda rota mantığını bozmadan, gerçek araç konumunu da görselleştirmeye imkân tanır.

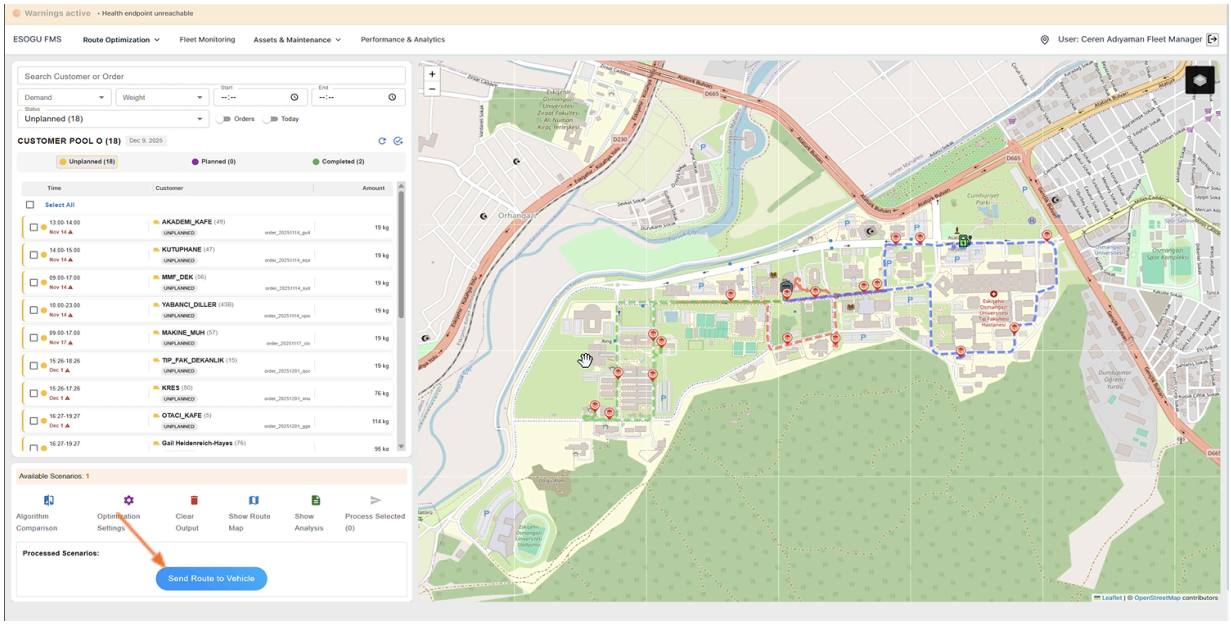
4.2 RouteVehicleSelector – Araç Seçimi ve A* Tetikleme Mantığı

Araç seçimi sırasında frontend, FIWARE üzerinden gelen araç konumlarını alır ve seçilen aracın depoda olup olmadığını kontrol eder. Eğer araç depoda ise, SUIT senaryosu gereği ek bir rota hesaplamasına ihtiyaç duyulmaz. Araç depoda değilse, frontend otomatik olarak Route Service'e A* rota hesaplama isteği gönderir. Bu istek, aracın mevcut konumu, heading bilgisi ve sabit depo koordinatlarını içerir. Böylece araçtan depoya doğru bir A* rotası hesaplanır ve sonuç Route4Vehicle.json dosyasına eklenir.

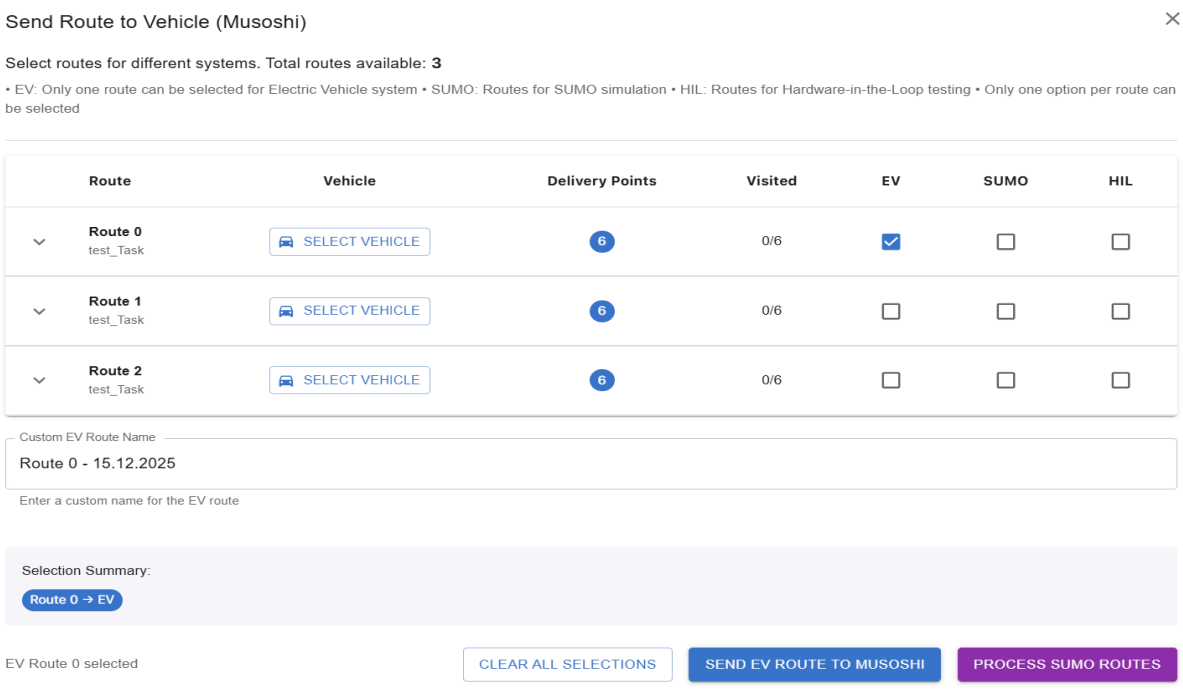
Bu işlem tamamen frontend tarafından tetiklenir ancak rota hesaplama ve JSON güncelleme işlemleri backend üzerinden gerçekleştirilir. Frontend, güncellenmiş Route4Vehicle.json dosyasını yeniden yükleyerek, harita ve tablo görünümünü senkronize eder. Bu sayede kullanıcı, yaptığı araç seçiminin rota üzerindeki etkisini anında görebilir.

Bu akışın frontend tarafındaki karşılığı Şekil 1, Şekil 2 ve Şekil 3'te gösterilmektedir. Şekil 1'de yer alan "Send Route to Vehicle" butonuna basıldığında, rotanın atanacağı aracın seçilmesini sağlayan ara ekran açılmakta ve bu adım Şekil 2'de sunulmaktadır. Şekil 2 üzerinden Vehicle butonunun seçilmesiyle birlikte, araç listesinin görüntülendiği Şekil 3 ekranına geçilmektedir. Bu ekranda araçlar mevcut konumlarıyla birlikte listelenmekte; depo konumunda bulunan araçlar "At Depot" etiketiyle işaretlenmekte ve yalnızca depo dışında bulunan araçlar için A* rota hesaplamasının tetikleneceği kullanıcıya açık bir şekilde bildirilmektedir.

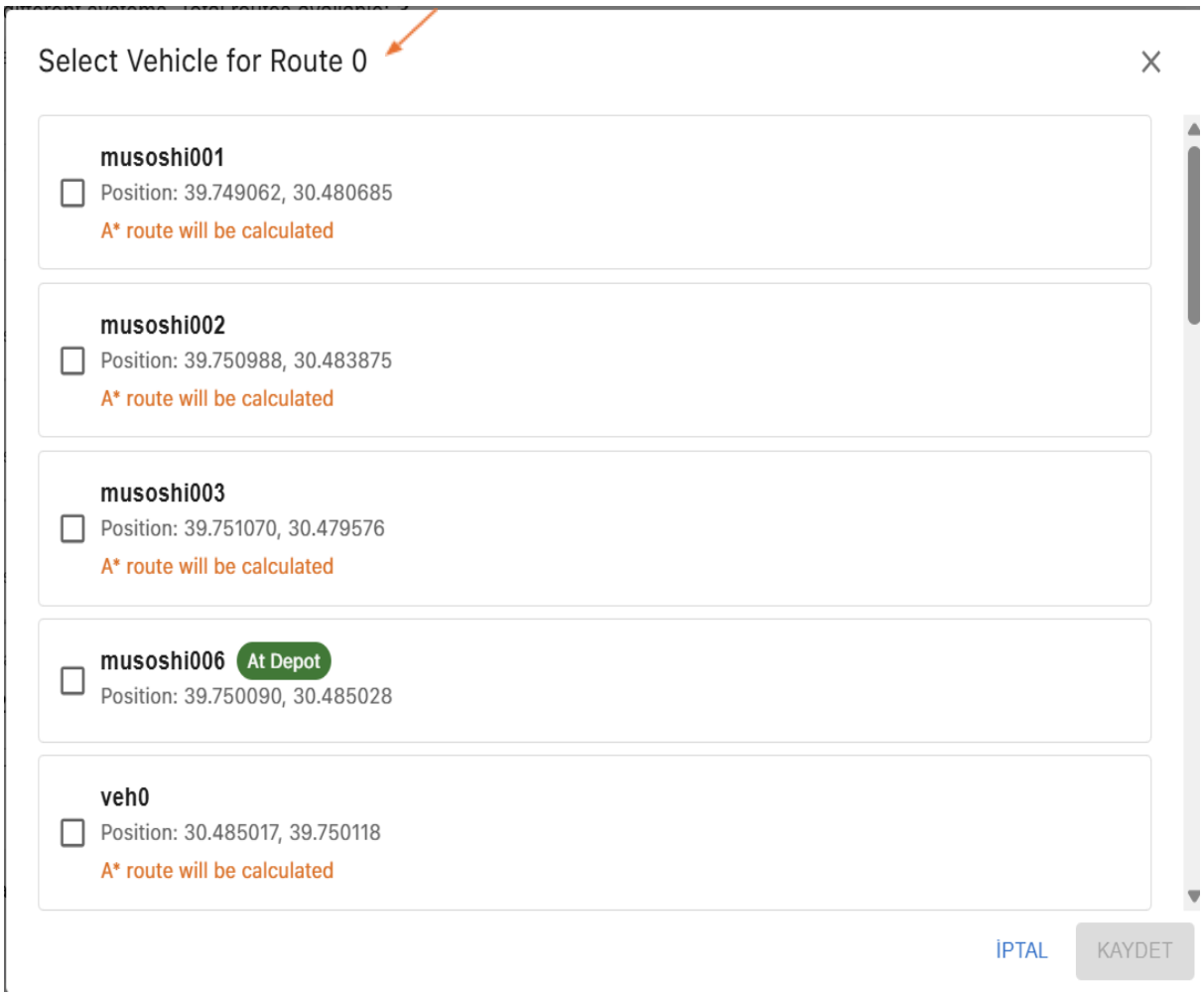
Şekil 1



Şekil 2



Şekil 3



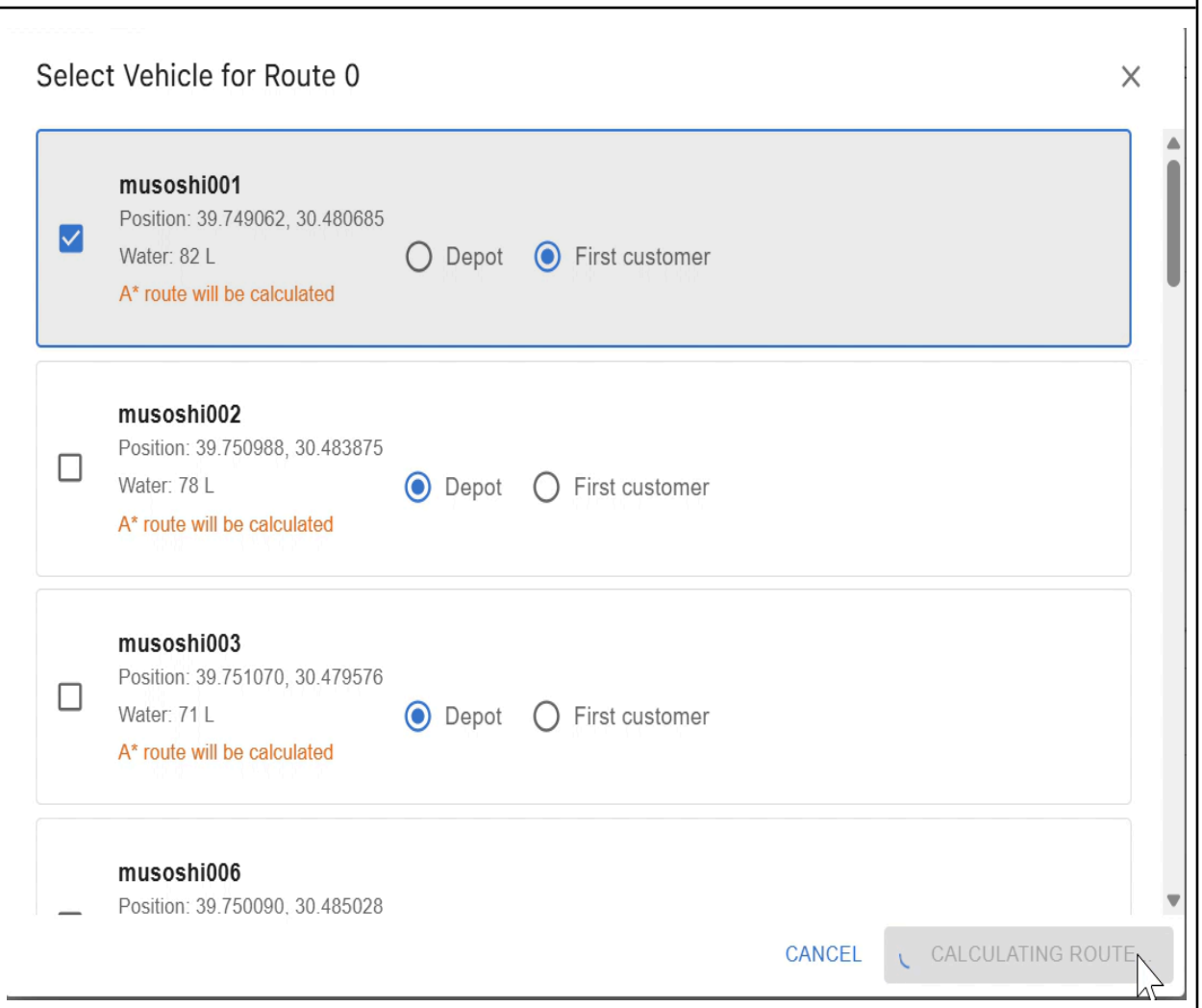
5. OPEVA Frontend

OPEVA projesinde frontend katmanı, rota–araç eşleştirme sürecini yalnızca görselleştiren bir arayüz değil; aynı zamanda operasyonel kararların alındığı aktif bir kontrol katmanı olarak tasarlanmıştır. SUIT senaryosundan farklı olarak OPEVA'da araçlar her zaman depoya yönlendirilmez. Araçlar, mevcut durumlarına ve kullanıcı tercihine bağlı olarak ya depoya ya da rotadaki ilk ziyaret edilmemiş müşteriye yönlendirilebilir. Bu nedenle OPEVA frontend'i, rota seçimiyle birlikte hedef tipi seçimini de kullanıcıya sunar ve bu seçimi routing sürecinin merkezine yerleştirir.

5.1 Araç Seçimi ve Hedef Tercihi Mantığı

OPEVA frontend'inde kullanıcı, bir rota için araç seçimi yaptığında sistem yalnızca aracı belirlemez; aynı zamanda bu araç için routing hedefinin ne olacağını da belirler. Araç seçim dialog'u açıldığında, FIWARE üzerinden gelen araç listesi yüklenir. Bu bilgiler, kullanıcının bilinçli bir seçim yapabilmesi için arayüzde açıkça sunulur. Her araç için kullanıcıya iki farklı hedef seçeneği sunulur: depo ve ilk müşteri. İlk müşteri seçeneği yalnızca ilgili rotada geçerli ve konumu bilinen bir teslimat noktası varsa aktif hale gelir. Böylece frontend, backend'e veya Route Service'e anlamsız bir hedef isteği göndermeden önce temel doğrulamayı yapmış olur. Bu yaklaşım, hatalı routing çağrılarını en baştan engeller. OPEVA senaryosunda SUIT senaryosuna ek, Şekil 5'te görüldüğü her araç için depo veya ilk müşteri hedefi ayrı ayrı seçilebilmekte olmasıdır.

Şekil 4



5.2 Araç Konumuna Bağlı A* Tetikleme Kararı

OPEVA senaryosunda A* algoritmasının çalıştırılıp çalıştırılmayacağı, yalnızca hedef tipine değil, aracın mevcut konumuna da bağlıdır. Frontend, seçilen aracın depoya olan mesafesini Haversine hesaplaması ile kontrol eder. Eğer hedef depo seçilmişse ve araç halihazırda depo konumundaysa, A* rota hesaplaması yapılmaz. Bu durumda araç “hazır” kabul edilir ve gereksiz hesaplama maliyetinden kaçınılır. Buna karşılık, hedef ilk müşteri olarak seçilmişse, araç depoda olsa dahi A* hesaplaması mutlaka tetiklenir. Çünkü bu senaryoda amaç, aracın mevcut konumundan ilk müşteriye doğrudan ve dinamik bir rota üretmektir. Bu ayrım, OPEVA’nın operasyonel gerçekçiliğini sağlayan temel kararlardan biridir.

5.3 A* İsteğinin Oluşturulması ve Gönderilmesi

A* hesaplamasının gerekli olduğu durumlarda frontend, Route Service’e gönderilecek isteği bağlama uygun şekilde hazırlar. Bu istek; aracın mevcut konumu, varsa önceki konumundan hesaplanan heading bilgisi, hedef koordinatları, hedef etiketi ve hedef tipi (depot veya first_customer) bilgilerini içerir. Eğer hedef ilk müşteri ise, ilgili müşteri bilgileri de isteğe eklenir. Bu yapı sayesinde Route Service, yalnızca “nereden nereye” sorusunu değil, hangi bağlamda bu rotayı hesapladığını da bilir. Bu bağlam bilgisi, backend tarafında Route4Vehicle.json güncellenirken start_point yapısının nasıl oluşturulacağını doğrudan etkiler.

5.4 Route4Vehicle.json Güncellemesi ve Senkronizasyon

Route Service’ten başarılı bir A* cevabı alındıktan sonra, frontend bu çıktıyı backend’deki A* entegrasyon endpoint’ine iletir. Backend, bu bilgiyi kullanarak Route4Vehicle.json dosyasında ilgili rotanın start_point alanını günceller. Hedef ilk müşteri ise, start_point tamamen aracın mevcut konumuna göre yeniden tanımlanır ve müşteri bağlamı eklenir. Hedef depo ise, mevcut rota yapısı korunarak yalnızca araçtan depoya olan başlangıç segmenti güncellenir. Frontend, bu işlem tamamlandıktan sonra Route4Vehicle.json dosyasını yeniden yükler ve harita ile tablo görünümünü günceller. Böylece kullanıcı, yaptığı hedef ve araç seçiminin rota üzerindeki etkisini anında görebilir.